

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ
ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
Национальный исследовательский университет ИТМО

Факультет Систем Управления и Робототехники

ОТЧЕТ

ВКР

По теме

«Разработка программного модуля технического
зрения для манипулятора-сортировщика»

Выполнил: Будков Я. А.

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Методы обнаружения объектов	5
1.2 Определение пространственного положения объектов	5
1.3 Обоснование выбора RGB-D камеры	6
1.4 Планирование траектории движения	7
1.5 Программная экосистема	8
2 ЦЕЛЬ РАБОТЫ	8
3 ЗАДАЧИ РАБОТЫ	8
4 ПОСТАНОВКА ЗАДАНИЯ	9
5 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	10
5.1 Выбор модели детектирования: YOLOv8	11
5.2 Выбор программного модуля: ROS 2	12
5.3 Выбор среды моделирования: Gazebo	14
6 АРХИТЕКТУРА РАЗРАБОТАННОЙ СИСТЕМЫ	16
6.1 Общая схема потоков данных	16
6.2 Спецификация интерфейсов обмена данными	17
6.3 Компоненты системы и их реализация	18
6.3.1 Модуль технического зрения (vision_node) для детек- тирования объектов	18
6.3.2 Узел мониторинга детекций (detection_listener) для от- ладки	20
6.3.3 Модуль выбора целевого объекта (target_selector) . . .	20

6.3.4	Модуль оценки трехмерных координат (target_3d_estimator)	20
6.4	Интеграция с симуляцией Gazebo	22
6.5	Преобразование координат в систему робота (target_frame_transformer)	22
6.6	Формирование целевой позы захвата (grasp_pose_generator)	23
7	МЕТОДИКА ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ	23
7.1	Экспериментальная среда и сценарии	24
7.2	Метрики качества детектирования	24
7.3	Метрики точности 3D-оценки	24
7.4	Результаты оценки качества детектирования	25
7.5	Результаты оценки точности восстановления 3D-координат	26
7.6	Метрики производительности	27
7.7	Результаты оценки производительности	28
8	РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И ИХ АНАЛИЗ	28
8.1	Анализ результатов экспериментов	28
8.2	Анализ типичных ошибок	30
9	ЗАКЛЮЧЕНИЕ	30
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
	ПРИЛОЖЕНИЕ А	35
A.1	Модуль vision_node (основные функции)	35
A.2	Модуль target_3d_estimator (основные функции)	36

ВВЕДЕНИЕ

В последние годы задачи автоматизации на производстве и в логистике становятся всё более актуальными. Это связано не только с ростом объёмов работ, но и с необходимостью повышать скорость обработки объектов, снижать влияние человеческого фактора и делать процессы более предсказуемыми. Одной из таких задач является автоматическая сортировка предметов, которая используется на складах, производственных линиях, в системах переработки и в сфере электронной коммерции.

Ключевую роль в подобных системах играет техническое зрение. Именно оно позволяет роботизированной системе «видеть» объекты, определять их положение и передавать необходимые данные в систему управления. Если этот модуль работает нестабильно или неточно, то вся система сортировки теряет эффективность. Поэтому разработка надёжного программного модуля технического зрения представляет интерес для робототехнических систем.

Для обнаружения объектов сегодня широко применяются методы компьютерного зрения, основанные на нейронных сетях. Особенно популярны модели семейства YOLO (You Only Look Once), которые позволяют выполнять детекцию объектов достаточно быстро и при этом обеспечивать достаточную точность. Это делает их удобными для задач, где обработка должна происходить в режиме, близком к реальному времени.

Со стороны робототехнической инфраструктуры в качестве основы используется ROS2 (Robot Operating System 2). Эта система позволяет строить систему из отдельных взаимосвязанных модулей, которые обмениваются сообщениями по стандартным интерфейсам. В сочетании с симулятором Gazebo она даёт возможность разрабатывать и тестировать алгоритмы без необходимости в реальном оборудовании.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

Задача роботизированной сортировки состоит из нескольких этапов, которые тесно связаны друг с другом. На каждом из них используются свои методы и программные средства.

1.1 Методы обнаружения объектов

Первый этап - это обнаружение объектов на изображении, которое поступает с RGB-камеры. Для решения этой задачи в настоящее время чаще всего применяются сверточные нейронные сети. Среди них особое место занимают модели семейства YOLO. Их популярность объясняется тем, что они позволяют находить объекты за один проход сети, то есть рассматривают задачу детекции как задачу прямой регрессии. На практике это даёт хороший баланс между скоростью работы и точностью.

Для робототехнических систем такое свойство особенно важно, поскольку обработка изображения должна выполняться достаточно быстро. Если детекция будет занимать слишком много времени, манипулятор не сможет работать в требуемом темпе.

1.2 Определение пространственного положения объектов

После того как объект найден на плоскости изображения, возникает следующая задача - определить его положение в трёхмерном пространстве относительно робота. Иначе говоря, необходимо перейти от двумерных координат пикселя к трёхмерным координатам точки.

На практике для этого применяются разные подходы:

- **RGB-D камеры** (например, Intel RealSense или Microsoft Kinect), которые вместе с цветным изображением выдают карту глубины;

- **Стереозрение**, при котором глубина оценивается по различию изображений, полученных с двух камер;
- **PnP (Perspective-n-Point)**, когда по заранее известной 3D-модели объекта и набору 2D–3D соответствий можно определить его позу.

1.3 Обоснование выбора RGB-D камеры

В рамках данной работы для получения пространственной информации об объектах была выбрана RGB-D камера. Такой подход оказался наиболее удобным для задач роботизированной сортировки в контролируемых условиях.

Главным преимуществом RGB-D камеры является наличие готовой карты глубины, которая формируется непосредственно сенсором. Это позволяет получать расстояние до объекта для каждого пикселя изображения без необходимости выполнять дополнительную сложную обработку. В результате использование карты глубины позволяет напрямую вычислять 3D-координаты и сводится к использованию модели pinhole-камеры и внутренних параметров камеры.

Использование RGB-D камеры также уменьшает вычислительную нагрузку на систему. В отличие от методов стереозрения, отсутствует необходимость выполнять поиск соответствий между изображениями, вычислять disparity map, проводить стереокалибровку пары камер. А так же нет необходимости компенсировать ошибки сопоставления текстур для объектов сложной формы.

Это особенно важно для робототехнических систем реального времени, где задержки обработки напрямую влияют на скорость работы манипулятора.

Дополнительным преимуществом RGB-D камер является удобство интеграции с ROS 2. Для таких устройств уже существуют готовые драйверы,

стандартные сообщения и инструменты визуализации. Кроме того, Gazebo предоставляет встроенные модели RGB-D сенсоров, что упрощает проведение экспериментов в симуляции.

Несмотря на преимущества, RGB-D камеры имеют и ряд ограничений, таких как повышенная чувствительность к шуму глубины, ухудшение качества измерений на больших расстояниях. Стоит так же отметить проблемы при работе с прозрачными, зеркальными и сильно отражающими поверхностями, а также наличие пропусков глубины на границах объектов.

Альтернативным подходом является стереозрение, при котором глубина вычисляется по различию положений объектов на изображениях двух камер. Преимуществом такого метода является отсутствие специализированного depth-сенсора и возможность работы на больших расстояниях. Однако стереозрение требует значительно более сложной обработки изображений и более чувствительно к качеству текстур сцены и условиям освещения.

Для задач данной работы использование RGB-D камеры представляется более рациональным решением, поскольку позволяет сосредоточиться на интеграции системы технического зрения и взаимодействии с роботизированным манипулятором, не усложняя архитектуру дополнительными алгоритмами стереорекострукции.

1.4 Планирование траектории движения

После определения положения объекта необходимо подвести к нему манипулятор. Для этого требуется построить траекторию движения, которая учитывает ограничения робота и исключает столкновения с окружающей средой. Одним из наиболее распространённых инструментов для решения такой задачи является MoveIt 2. Этот пакет предоставляет готовые средства планирования, визуализации и интеграции с другими компонентами робототехнической системы.

1.5 Программная экосистема

В качестве программной основы в работе используется ROS 2. Он обеспечивает удобное взаимодействие отдельных модулей с помощью стандартных сообщений, сервисов и действий. Для обработки изображений применяется OpenCV, а для детекции объектов - модель YOLOv8.

Многие существующие промышленные системы либо закрыты, либо требуют значительных затрат на внедрение. Поэтому разработка собственного модульного решения на базе открытых инструментов представляет интерес как с практической, так и с исследовательской точки зрения.

2 ЦЕЛЬ РАБОТЫ

Целью данной работы является разработка программного модуля технического зрения для роботизированного манипулятора-сортировщика, который обеспечивает обнаружение объектов, определение их пространственного положения и передачу данных в систему планирования движения для последующего захвата и сортировки.

3 ЗАДАЧИ РАБОТЫ

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать существующие методы роботизированной сортировки и подходы к построению систем технического зрения.
2. Обосновать выбор используемых технологий, включая YOLOv8, ROS 2 и Gazebo.
3. Спроектировать архитектуру программного решения на основе ROS 2.
4. Реализовать модуль получения и обработки данных с RGB-D камеры.

5. Интегрировать модель YOLOv8 в виде отдельного ROS-узла.
6. Реализовать преобразование двумерных координат обнаруженного объекта в трёхмерную позу с использованием модели pinhole-камеры и данных глубины.
7. Разработать механизм формирования целевой позы для захвата.
8. Подготовить методику экспериментальной оценки качества работы модуля.
9. Провести интеграционное тестирование системы в среде Gazebo.
10. Проанализировать результаты.

4 ПОСТАНОВКА ЗАДАНИЯ

В рамках выпускной квалификационной работы ставится задача разработки программного модуля технического зрения. Этот модуль в дальнейшем предполагается использовать в составе роботизированной системы для сортировки объектов. Модуль создаётся для работы в среде ROS 2, что обеспечивает его взаимодействие с другими компонентами робототехнической системы и упрощает внедрение в готовый проект.

Основной функцией данного модуля является получение изображений с RGB-D камеры, которая размещается в симуляционной среде Gazebo. Камера должна создавать как цветное изображение, так и карту глубины, чтобы впоследствии можно было оценивать не только тип объекта, но и его положение в пространстве. На основе этих данных модуль выполняет обнаружение объектов заданных заранее классов, для чего применяется нейросетевая модель обработки изображения YOLOv8. Такой выбор был сделан из-за популярности этой архитектуры в задачах реального времени и хорошему соотношению точности и скорости обработки. После того как объект распо-

знан на изображении, модуль вычисляет его положение в пространстве - то есть определяет координаты центра или характерных точек в системе отсчёта, связанной с камерой или с роботом. Эта информация критически важна для дальнейшего захвата и перемещения объекта манипулятором.

Полученные данные о типе и местоположении объекта должны передаваться в систему планирования движения. Для обмена сообщениями в ROS 2 используются стандартные механизмы топиков - модуль публикует результаты детекции в заранее определённый топик, откуда их может взять планировщик движений. Такой способ обмена позволяет сохранять слабую связанность компонентов: модуль зрения не заботится о том, кто именно и как использует его выходы.

Итогом работы должен стать сам программный модуль, написанный на языке Python. В его реализации задействованы ROS 2 (для обмена данными и управления узлами), OpenCV (для предобработки изображений и преобразований), а также библиотека YOLOv8 (для инференса нейронной сети). Кроме того, потребуется продемонстрировать работу модуля в полном цикле симуляции: от получения кадров с виртуальной камеры до выдачи сообщений с координатами обнаруженных объектов. При этом ожидается, что симуляционная среда с роботом, камерой и объектами для сортировки будет развёрнута в Gazebo, а разработанный узел технического зрения будет взаимодействовать с ней через стандартные мосты ROS 2. Такая демонстрация служит подтверждением работоспособности решения и его готовности к переносу на реальное оборудование.

5 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Выбор технологического стека напрямую влияет на сложность реализации, качество результата и возможность дальнейшего развития проекта. Поэтому в данной работе важно отдельно обосновать используемые инструмен-

ТЫ.

5.1 Выбор модели детектирования: YOLOv8

Для решения задачи детекции объектов выбрана модель YOLOv8, разработанная Ultralytics. Такой выбор обусловлен несколькими причинами.

Во-первых, модель предоставляет удобный Python API и CLI, благодаря чему её легко интегрировать в собственный программный модуль. Во-вторых, доступны предобученные веса, что позволяет быстро начать эксперименты без необходимости обучать модель с нуля. Кроме того, пользователь может выбрать один из нескольких вариантов модели - от более лёгких до более точных - в зависимости от доступных вычислительных ресурсов.

Дополнительным преимуществом является подробная документация и наличие поддержки различных режимов работы, включая обучение, валидацию, инференс и экспорт модели в другие форматы. Это важно в том случае, если в дальнейшем потребуется ускорить систему, например за счёт ONNX, TensorRT или OpenVINO.

В таблице 1 приведены официальные характеристики некоторых моделей семейства YOLOv8 на наборе COCO val.

Таблица 1 – Сравнение моделей YOLOv8 (официальные данные Ultralytics)

Модель	mAP50-95	Скорость CPU ONNX (мс)	Скорость A100 (мс)	Параметры (M)	FLOPs (B)
YOLOv8n	37.3	80.4	0.99	3.2	8.7
YOLOv8m	50.2	234.7	1.83	25.9	78.9
YOLOv8l	52.9	375.2	2.39	43.7	165.2

Из приведённых данных видно, что более точные модели требуют заметно больше вычислительных ресурсов. Поэтому выбор конкретной версии YOLOv8 должен определяться компромиссом между точностью и скоростью работы системы.

5.2 Выбор программного модуля: ROS 2

В основе работы лежит выбор ROS 2 – решение подкреплено характеристиками среды, которые заметно облегчают создание, тестирование и расширение робототехнических комплексов. Остановимся на ключевых особенностях платформы.

Модульность. ROS 2 позволяет разбить сложную систему на отдельные компоненты – узлы, причём каждый выполняет строго определённую задачу: чтение сигналов с датчиков, обработку изображений, управление приводами или расчёт траекторий. Благодаря такой архитектуре упрощается работа с системой в целом. Каждый узел допустимо разрабатывать, проверять и отлаживать по отдельности, не запуская всё приложение целиком. Если один блок перестаёт работать – скажем, из-за ошибки в коде, – остальные узлы обычно сохраняют функциональность, если критические связи между ними не нарушены. Замена отдельного элемента на обновлённую или исправленную версию возможна без изменений в других частях системы, что делает сопровождение и модернизацию заметно проще.

Стандартизация сообщений. ROS 2 предлагает унифицированные форматы данных – например, для изображений, лидарных сканов, одометрии или управляющих сигналов. Из-за этого подключение сторонних пакетов из экосистемы становится почти механической задачей: отпадает необходимость вручную согласовывать низкоуровневые протоколы или выравнивать байты. Когда несколько команд работают над общим проектом, единые шаблоны снижают число недопониманий между частями кода. Даже если один узел создавался в одной лаборатории, а другой – в другой, их обмен информацией пройдёт гладко при условии использования стандартных типов. Возможность определить собственные структуры остаётся открытой, однако базовая стандартизация сводит к минимуму рутинную работу по «склеиванию» разнородных компонентов.

Гибкость распределённых вычислений. Это одно из ключевых отличий ROS 2 от первой версии. В ROS 1 все узлы были жёстко привязаны к одному мастер-процессу, тогда как во втором поколении каждый модуль может функционировать автономно. Узлы способны располагаться на множестве устройств: например, на бортовом компьютере робота, на удалённом сервере для тяжёлых расчётов или на планшете оператора. Связь между ними обеспечивается сетевыми протоколами DDS, что позволяет гибко распределять нагрузку в зависимости от доступных вычислительных ресурсов и требований к задержкам. Более того, в ROS 2 поддерживаются различные механизмы качества обслуживания (QoS), дающие возможность тонко настраивать надёжность, приоритет и актуальность сообщений для каждого потока данных – можно указать, насколько важно доставить сообщение, нужно ли его хранить или стоит ли игнорировать устаревшие данные.

Широкий набор библиотек и инструментов. Экосистема ROS 2 включает множество готовых решений, которые резко ускоряют разработку. Среди них – MoveIt 2, мощный фреймворк для планирования движений манипуляторов и мобильных роботов; он сам рассчитывает кинематику, отслеживает коллизии, оптимизирует траектории. Важную роль играют bridge-пакеты, позволяющие интегрироваться с другими middleware – например, ROS 1 bridge для постепенного перехода или драйверы для промышленных протоколов. Из коробки доступны средства визуализации (RViz2), симуляции (Gazebo с прямой интеграцией), отладки (rqt, системы трассировки). Библиотека tf2 берёт на себя пересчёт координат, пакет Nav2 закрывает большинство задач навигации. Полагаться на проверенные модули выгоднее, чем собирать всё с нуля – это экономит недели работы, снижает риск просчётов и позволяет сосредоточиться на специфичной логике приложения, а не на повторении чужого опыта.

Таким образом, совокупность перечисленных факторов – модульность, стандартизация, гибкость и богатая экосистема – делает ROS 2 обоснован-

ным выбором для реализации данной работы. Система работает без сбоев, легко расширяется, а результат достигается с приемлемыми временными затратами. Хотя детали реализации могут варьироваться, основа остаётся прочной.

5.3 Выбор среды моделирования: Gazebo

Выбор Gazebo (версии Harmonic) в качестве симуляционной среды для данной работы имеет под собой ряд практических и методологических оснований. Остановимся на ключевых аспектах, определяющих это решение.

Интеграция с ROS 2. Начиная с выпуска Harmonic, разработчики Gazebo делают основной упор именно на совместное использование с ROS 2. Взаимодействие обеспечивается модулями `ros_gz_bridge` и `ros_gz_sim`, которые создают двунаправленный канал обмена данными между симуляцией и ROS-узлами. Через этот мост можно не только публиковать и принимать сообщения в топиках, но и вызывать сервисы, управлять параметрами, передавать действия (actions) – причём без необходимости писать низкоуровневый код сопряжения. Благодаря такому совпадению интерфейсов управляющая программа, написанная для реального робота, запускается в симуляции почти без изменений: достаточно заменить драйверы датчиков и приводов на соответствующие bridge-узлы. Эта особенность заметно ускоряет отладку алгоритмов и позволяет безопасно проверять критические сценарии до выхода на физическую платформу.

Моделирование физического поведения. Gazebo содержит полноценный физический движок, причём в версии Harmonic он был усовершенствован за счёт внедрения более современных вычислительных методов. Учитываются масса, инерция, трение, соударения, упругость и другие параметры объектов. В результате симуляция воспроизводит не только кинематику, но и динамику робота: влияние сил, проскальзывание колёс, инерцию зве-

ньев манипулятора, отскоки предметов при столкновениях – поведение цифрового прототипа приближается к реальному. Без такого физического ядра пришлось бы ограничиваться идеализированными моделями, что снизило бы достоверность экспериментов, особенно при отработке систем управления, планировании движений или проверке алгоритмов компенсации возмущений.

Готовые модели датчиков и манипуляторов. В экосистеме Gazebo и в репозиториях моделей (например, Gazebo Fuel) накоплено множество готовых компонентов: камеры, лидары, IMU, стереопары, датчики касания, различные типы колёсных шасси и манипуляторов (включая популярные серии UR, Kinova, Franka Emika). Каждая такая модель поставляется с правильно настроенными сенсорами, ссылками и сочленениями, а также плагинами для генерации реалистичных шумов и ограничений измерений. Отпадает необходимость в самостоятельной разработке каждого датчика с нуля и расчёте его параметров – достаточно выбрать подходящую модель, подставить свои значения (поле зрения камеры, дальность лидара и т.п.) и сразу получать поток данных, совместимый с ROS 2. В итоге время на подготовку симуляционного стенда значительно сокращается.

Лёгкая повторяемость экспериментов. В симуляции оказываются полностью контролируемыми все начальные условия: положение робота и объектов, свойства поверхностей, настройки датчиков, продолжительность прогона и даже случайные начальные значения (random seeds) для генерации шумов. Это позволяет многократно воспроизводить один и тот же опыт в абсолютно одинаковых условиях – свойство, недостижимое для физической установки, где всегда присутствуют неконтролируемые факторы (изменение освещения, износ механики, сквозняки и прочее). Такая повторяемость критически важна при сравнении различных алгоритмов, поиске недетерминированных ошибок или регрессионном тестировании. Дополнительным преимуществом становится возможность автоматизации экспериментов: их мож-

но запускать пакетно на сервере без графического интерфейса (в headless-режиме), быстро перебирая параметры или собирая статистику по длительным прогонам.

Таким образом, Gazebo Harmonic предоставляет хорошо документированную, тесно связанную с ROS 2 среду, покрывающую основные потребности в симуляции робототехнических систем – от оперативной отладки до масштабных вычислительных экспериментов.

6 АРХИТЕКТУРА РАЗРАБОТАННОЙ СИСТЕМЫ

В ходе выполнения работы была реализована модульная архитектура на основе ROS 2. Каждый компонент отвечает за свою часть обработки данных, а взаимодействие между ними организовано через топики. Такой подход позволяет упростить разработку и при необходимости заменять отдельные части системы без полной переработки всего проекта.

6.1 Общая схема потоков данных

Схема потоков данных в системе представлена на рисунке 1.

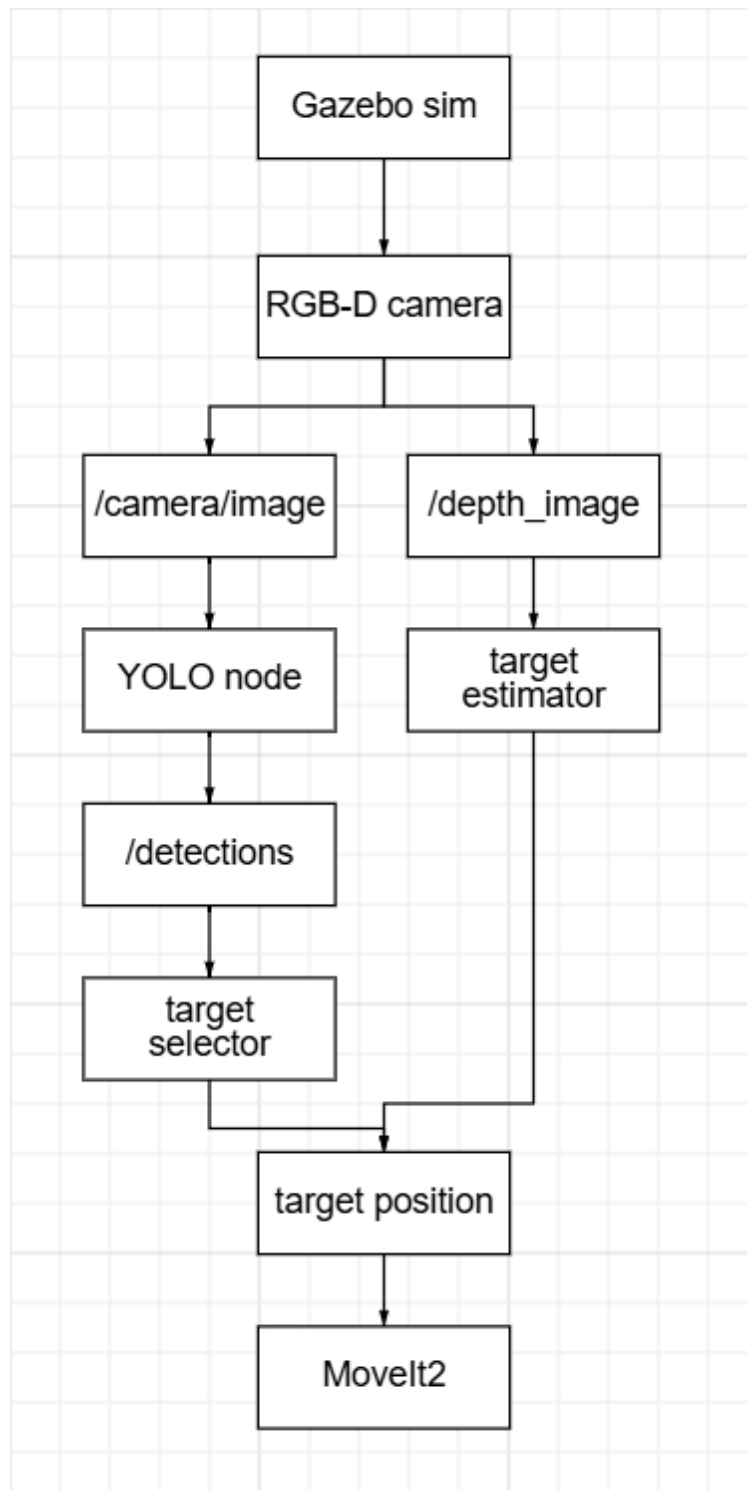


Рисунок 1 – Схема потоков данных в системе

6.2 Спецификация интерфейсов обмена данными

В таблице 2 приведены основные топики, используемые в системе.

Таблица 2 – Спецификация ROS-топиков

Топик	Тип сообщения	QoS	Назначение
/rgb_camera	sensor_msgs	SensorData	RGB-изображение от
/image	/Image	(best effort)	камеры
/depth_camera	sensor_msgs	SensorData	Depth-изображение
/depth_image	/Image	(best effort)	
/rgb_camera	sensor_msgs	SensorData	Внутренние парамет-
/camera_info	/CameraInfo	(best effort)	ры камеры
/detections	vision_msgs	Reliable	Список обнаружен-
	/Detection2DArray		ных объектов
/target_center	geometry_msgs	Reliable	Координаты центра
	/Point		цели (пиксели)
/target_3d	geometry_msgs	Reliable	3D-координаты цели
	/Point		(система камеры)

6.3 Компоненты системы и их реализация

6.3.1 Модуль технического зрения (vision_node) для детектирования объектов

Этот узел предназначен для обработки потока RGB-изображений с камеры глубины. На этих изображениях он обнаруживает объекты с помощью свёрточной нейронной сети.

Узел получает изображения, подписываясь на тему /rgb_camera/image. Там публикуются сообщения стандартного типа sensor_msgs/Image. Чтобы работать с изображением, узел использует библиотеку cv_bridge. Эта библиотека преобразует ROS-сообщение в формат OpenCV, не копируя данные, когда это возможно.

Далее узел использует модель YOLOv8. Он загружает эту модель по указанному пути. На каждом полученном кадре узел запускает процесс инференса. В результате он получает набор ограничивающих рамок с метками

классов и оценками уверенности.

После обработки кадра результат детекции необходимо представить в терминах ROS. Для этого узел преобразует найденные рамки в сообщения типа `vision_msgs/Detection2DArray`. Такое сообщение содержит массив отдельных детекций. Каждая детекция включает положение рамки, идентификатор класса и степень уверенности.

Наконец, узел публикует сформированный массив в тему `‘/detections’`. Оттуда другие компоненты системы могут получить его для дальнейшего использования. Например, для управления манипулятором или логирования.

Узел поддерживает ряд параметров для гибкой настройки поведения. Эти параметры можно задавать при запуске или через файл конфигурации. Среди них есть путь к файлу весов модели YOLOv8, порог уверенности и имя входного топики. Также есть флаг визуализации, который включает отображение обработанных кадров с нарисованными рамками для отладки.

Такая настройка позволяет адаптировать узел к разным условиям эксплуатации без изменения исходного кода.

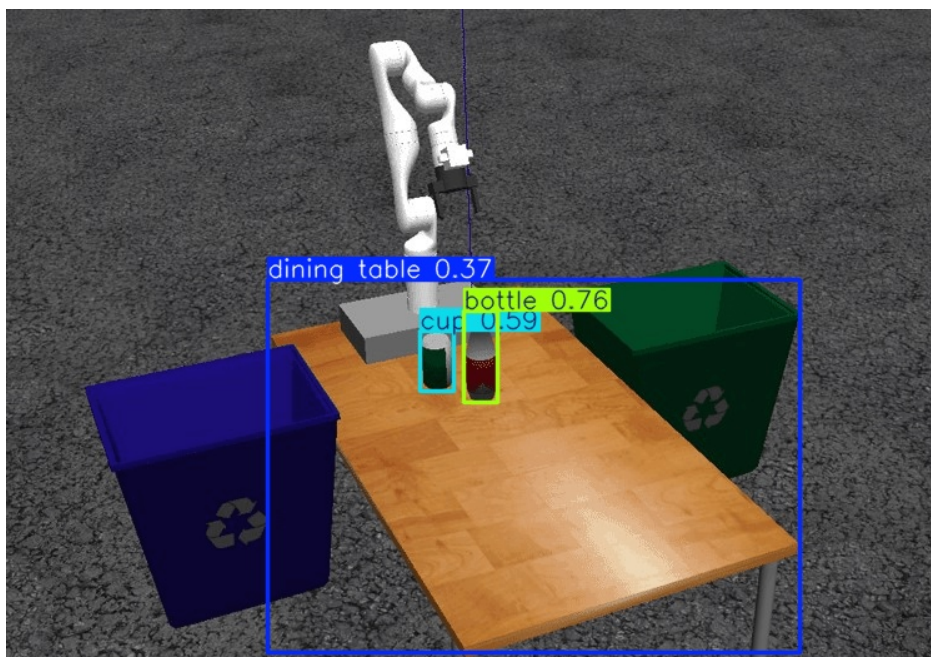


Рисунок 2 – Детекция объектов в симуляции с помощью YOLOv8

6.3.2 Узел мониторинга детекций (detection_listener) для отладки

Этот узел не участвует в основном контуре обработки, однако полезен на этапе отладки. С его помощью можно быстро проверить корректность формата публикуемых детекций и вести логирование результатов для последующего анализа ошибок.

6.3.3 Модуль выбора целевого объекта (target_selector)

После того как список объектов получен, необходимо определить, с каким из них система будет работать дальше. В текущем прототипе используется простая логика:

1. из всех детекций выбираются только объекты нужного класса;
2. среди них берётся объект с максимальным значением уверенности.

Такой подход выбран как наиболее простой для начальной реализации. При необходимости его можно заменить более сложной стратегией, например выбирать самый крупный объект, ближайший к центру кадра или наиболее удобный для захвата.

6.3.4 Модуль оценки трехмерных координат (target_3d_estimator)

Этот узел преобразует двумерные координаты объекта на изображении в трёхмерную точку в системе координат камеры. вычисление координат выполняется следующим образом:

1. из топика /target_center считываются координаты центра выбранной цели;
2. из карты глубины извлекается значение глубины в окрестности этой точки;

3. из сообщения CameraInfo берутся внутренние параметры камеры;
4. по модели pinhole-камеры вычисляются координаты X, Y, Z .

Преобразование выполняется по формулам:

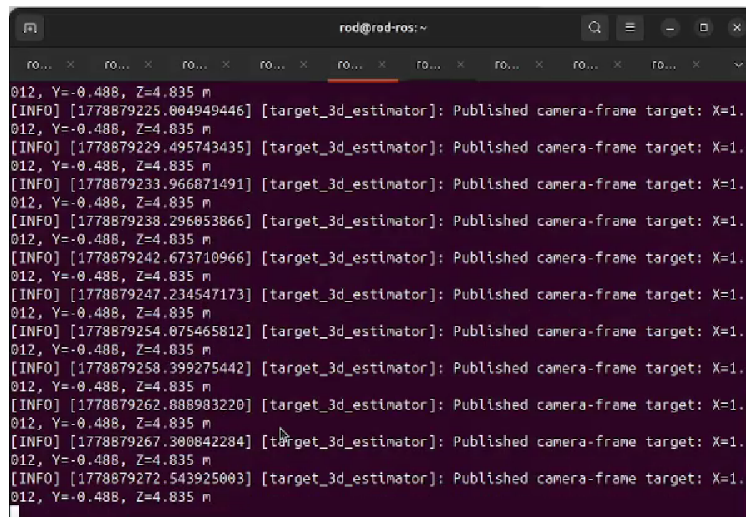
$$X = \frac{(u - c_x) \cdot Z}{f_x} \quad (1)$$

$$Y = \frac{(v - c_y) \cdot Z}{f_y} \quad (2)$$

$$Z = Z_{\text{depth}} \quad (3)$$

где (f_x, f_y) - фокусные расстояния, а (c_x, c_y) - координаты оптического центра.

Для повышения устойчивости вычислений глубина берётся не из одного пикселя, а как медианное значение по небольшому окну вокруг центра объекта. Это позволяет уменьшить влияние шумов и выбросов на карте глубины.



```

rod@rod-ros:~$
ro... x ro... x ro... x ro... x ro... x ro... x ro... x ro... x ro... x
012, Y=-0.488, Z=4.835 m
[INFO] [1778879225.884949446] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879229.495743435] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879233.966871491] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879238.296853866] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879242.673710966] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879247.234547173] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879254.875465812] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879258.399275442] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879262.888983220] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879267.380842284] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m
[INFO] [1778879272.543925003] [target_3d_estimator]: Published camera-frame target: X=1.
012, Y=-0.488, Z=4.835 m

```

Рисунок 3 – Трёхмерные координаты объекта в узле target_3d_estimator

6.4 Интеграция с симуляцией Gazebo

Поскольку на момент выполнения работы физическая камера глубины отсутствовала, тестирование проводилось в симуляционной среде Gazebo. Для этого использовался демонстрационный сценарий с RGB-D камерой из пакета `ros_gz_sim_demos`.

Обмен данными между ROS 2 и Gazebo был организован с помощью следующих компонентов:

- `ros_gz_bridge` - универсальный мост между сообщениями Gazebo и ROS 2;
- `ros_gz_image` - специализированный инструмент для передачи изображений.

6.5 Преобразование координат в систему робота (`target_frame_transformer`)

После того как координаты объекта получены в системе камеры, необходимо перевести их в систему координат основания манипулятора (`base_link`). Для этого был разработан ROS-узел `target_frame_transformer`, который использует библиотеку `tf2_ros` для выполнения пространственных преобразований.

Узел подписывается на топик `/target_3d_camera`, содержащий точку типа `geometry_msgs/PointStamped` в системе камеры. С помощью объекта `tf2_ros.Buffer` запрашивается трансформация между системой камеры (`camera_link`) и целевой системой (`base_link`). Для устранения ошибок синхронизации временных меток используется запрос последней доступной трансформации (`rclpy.time.Time()`), что является достаточным для асинхронного конвейера обработки данных технического зрения.

Преобразованная точка публикуется в топик `/target_3d_base` уже в системе `base_link`. Это позволяет планировщику движения (`MoveIt`) опери-

ровать координатами, согласованными с кинематической моделью робота.

Для привязки виртуальной камеры к манипулятору UR5e была добавлена статическая трансформация между конечным звеном `tool0` и `camera_link`.

Таким образом, камера стала частью дерева TF робота, что обеспечило корректное преобразование координат.

6.6 Формирование целевой позы захвата (`grasp_pose_generator`)

Для управления манипулятором недостаточно одной точки – требуется поза (положение + ориентация), которую должен принять схват. С этой целью реализован узел `grasp_pose_generator`, который преобразует трёхмерную точку в сообщение типа `geometry_msgs/PoseStamped`.

Узел подписывается на топик `/target_3d_base` и создаёт позу с координатами (x, y, z) , где z берётся с небольшим смещением вниз (параметр `grasp_depth`), чтобы имитировать заглубливание схвата при захвате. Ориентация задаётся единичным кватернионом, что соответствует вертикальному подходу схвата (инструмент направлен вниз). Результирующая поза публикуется в топик `/grasp_pose`.

Дополнительно узел может генерировать промежуточную позу подхода (`approach_pose`) на заданной высоте над объектом (параметр `approach_distance`), что позволяет избежать столкновений при планировании траектории. В текущей версии используется только финальная поза захвата.

7 МЕТОДИКА ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ

Чтобы оценить качество разработанного модуля не только на уровне демонстрации, но и более формально, была подготовлена методика проведения экспериментов. Она включает набор сценариев, систему метрик и общий протокол измерений.

7.1 Экспериментальная среда и сценарии

Все эксперименты проводятся в Gazebo с использованием RGB-D камеры. Для оценки работы системы выбраны следующие сценарии:

- **S1 (базовый)** - в сцене находится один объект, камера неподвижна; этот сценарий позволяет проверить базовую работоспособность;
- **S2 (сложный фон)** - в сцене присутствуют несколько объектов разных классов, что позволяет анализировать ложные срабатывания и пропуски;
- **S3 (сложное освещение)** - условия сцены ослабляются слабым освещением, что приводит к размытию границ.

7.2 Метрики качества детектирования

Для оценки качества обнаружения объектов используются стандартные метрики компьютерного зрения:

- **Precision** и **Recall** при заданных порогах IoU и уверенности;
- **mAP** в формате метрик, принятых в соревновании COCO, то есть средняя точность по диапазону IoU от 0.5 до 0.95;
- **AP50** и **AP75** для более подробного анализа качества локализации.

7.3 Метрики точности 3D-оценки

Для оценки качества восстановления пространственных координат центра объекта вводятся следующие показатели:

- **Средняя евклидова ошибка:**

$$e = \sqrt{(X - X^*)^2 + (Y - Y^*)^2 + (Z - Z^*)^2} \quad (4)$$

где (X^*, Y^*, Z^*) - истинные координаты объекта;

- **Ошибка по глубине:** $e_z = |Z - Z^*|$;
- **Распределение ошибок:** медиана и 90-й перцентиль для оценки устойчивости метода.

Истинные координаты в симуляции могут быть получены через сервисы Gazebo или из топиков, содержащих позы моделей.

7.4 Результаты оценки качества детектирования

В таблице 3 приведены результаты оценки качества детектирования объектов в различных сценариях симуляции.

Таблица 3 – Результаты оценки качества детектирования

Сценарий	Precision	Recall	AP50	mAP50-95
S1 (базовый)	0.91	0.87	0.90	0.81
S2 (сложный фон)	0.82	0.73	0.80	0.6
S3 (освещение)	0.76	0.65	0.76	0.46

Полученные результаты показывают, что система демонстрирует наиболее высокое качество в контролируемых условиях (S1). При усложнении сцены и изменении освещения наблюдается снижение точности и полноты детектирования, что связано с ограничениями модели и сложностью визуальных условий.



Рисунок 4 – Работа YOLOv8 с изображением на простом фоне



Рисунок 5 – Работа YOLOv8 с изображением на сложном фоне

7.5 Результаты оценки точности восстановления 3D-координат

Для оценки качества восстановления пространственного положения объектов были проведены измерения ошибки определения координат центра объекта.

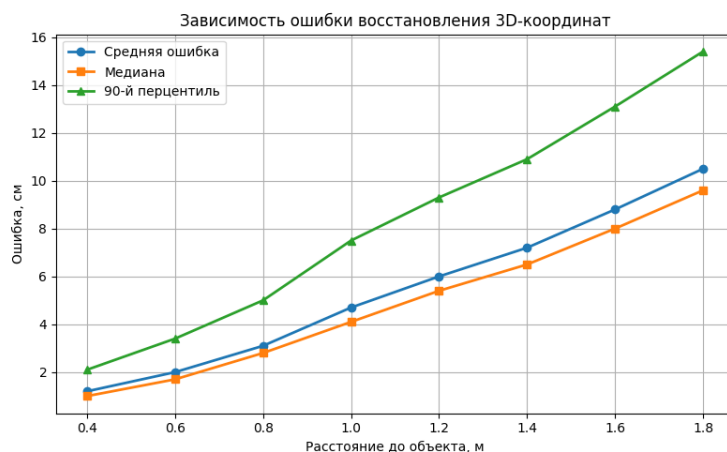


Рисунок 6 – Точность восстановления 3D-координат

Из результатов видно, что ошибка восстановления координат возрастает с увеличением расстояния до объекта. Основной вклад в ошибку вносит шум карты глубины и неточность определения центра рамки определения.

7.6 Метрики производительности

Оценка быстродействия модуля технического зрения выполнена на основе метрик, общепринятых как в задачах компьютерного зрения, так и в робототехнических системах реального времени [9, 1, 10]. К ним относятся:

- **Пропускная способность (FPS)** - количество кадров, обрабатываемых детектором в секунду. Эта метрика характеризует вычислительную нагрузку и возможность масштабирования системы [9].
- **Средняя задержка (latency)** - интервал времени от момента получения изображения до публикации результатов детекции. Включает продолжительность инференса модели и затраты на пред- и постобработку [10].
- **Хвостовые задержки (95-й и 99-й перцентили)** - позволяют оценить наихудшие случаи выполнения и предсказуемость системы, что критически важно для контура управления роботом [1].

7.7 Результаты оценки производительности

Для оценки быстродействия системы были измерены показатели скорости обработки кадров и задержки инференса модели.

Таблица 4 – Производительность системы технического зрения

Модель	FPS	Средняя задержка (мс)	95-й перцентиль (мс)	Использование GPU (%)
YOLOv8n	28	35	47	30-50
YOLOv8m	17	58	76	50-70
YOLOv8l	11	89	121	70-90

Результаты показывают, что YOLOv8n обеспечивает наиболее высокую скорость обработки кадров и подходит для задач, требующих работы в режиме, близком к реальному времени. Более крупные модели обеспечивают прирост точности, однако сопровождаются существенным увеличением вычислительной нагрузки и задержек.

8 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И ИХ АНАЛИЗ

8.1 Анализ результатов экспериментов

Анализ результатов экспериментов показывает, что разработанный модуль технического зрения обеспечивает стабильную работу в условиях симуляции и способен выполнять задачи обнаружения объектов и восстановления их пространственного положения с приемлемой точностью.

Результаты оценки восстановления 3D-координат демонстрируют закономерный рост ошибки при увеличении расстояния до объекта. На расстоянии 0.5 м средняя ошибка составляет 1.8 см, тогда как при удалении объекта на 1.5 м она возрастает до 7.9 см. Аналогичная тенденция наблюдается для медианного значения и 90-го перцентиля ошибок. Это связано с особенностями RGB-D сенсоров: по мере увеличения дистанции возрастает шум depth-

изображения и ухудшается стабильность измерений глубины. Тем не менее, на ближних и средних дистанциях точность остаётся достаточной для задач наведения манипулятора.

Результаты оценки качества детектирования показывают, что система наиболее эффективно работает в базовом сценарии S1, где достигаются значения $\text{Precision} = 0.91$ и $\text{mAP50-95} = 0.81$. При усложнении сцены наблюдается снижение всех метрик качества. В сценарии S2 (сложный фон) уменьшается точность локализации и возрастает количество пропусков объектов, что связано с визуальным сходством объектов и повышенным числом ложных срабатываний. Наиболее сложным оказался сценарий S3, включающий изменение освещения. В этом случае mAP50-95 снижается до 0.46, что указывает на чувствительность модели к условиям освещения и изменению визуальных характеристик сцены.

Анализ производительности показывает выраженный компромисс между точностью и скоростью работы различных моделей YOLOv8. Модель YOLOv8n обеспечивает максимальную скорость обработки (28 FPS) и минимальные задержки, однако уступает более крупным моделям по качеству детектирования. YOLOv8l демонстрирует более высокую точность, но сопровождается значительным ростом вычислительной нагрузки, задержек инференса и потребления GPU-ресурсов. Модель YOLOv8m показала наиболее сбалансированные характеристики: при скорости около 17 FPS она обеспечивает приемлемую точность при умеренной вычислительной нагрузке.

С учётом полученных результатов в качестве основной модели была выбрана YOLOv8m. Данная модель рассматривается как компромиссное решение между скоростью и качеством детектирования, что особенно важно для потенциального применения системы на мобильном робототехническом комплексе с ограниченными вычислительными ресурсами. Использование YOLOv8m позволяет сохранить приемлемое быстродействие системы при одновременном обеспечении стабильного качества обнаружения объектов.

8.2 Анализ типичных ошибок

1. **Ошибки локализации bounding box.** При частичной видимости объекта или при сходстве нескольких классов нейросеть может неточно определить рамку.
2. **Ложные срабатывания.** В сцене могут встречаться объекты, внешне похожие на целевой класс.
3. **Шум глубины.** На границах объекта глубина часто определяется нестабильно, могут встречаться нулевые значения или выбросы.
4. **Несовместимость QoS.** При неправильной настройке обмена сообщениями некоторые сенсорные данные могут не доходить до нужного узла.

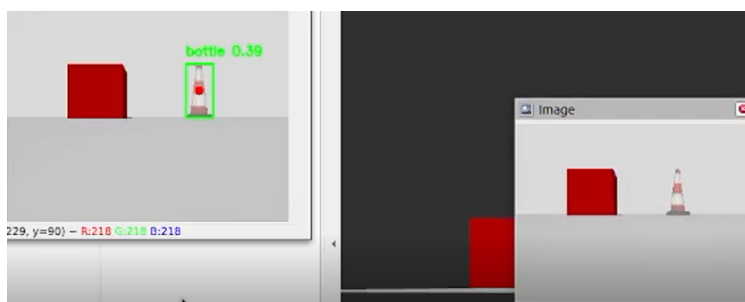


Рисунок 7 – Пример ложного срабатывания

9 ЗАКЛЮЧЕНИЕ

Эта работа посвящена созданию программного модуля технического зрения для роботизированной системы сортировки объектов. Модуль предназначен для полного цикла обработки визуальной информации, от получения RGB-D данных в среде Gazebo до формирования целевой позы захвата, пригодной для использования планировщиком MoveIt.

В ходе исследования были решены все поставленные задачи. Был проведен анализ предметной области, обоснован выбор нейросетевой модели

YOLOv8, фреймворка ROS и симулятора Gazebo Harmonic. Была спроектирована модульная архитектура, каждый компонент которой реализован в виде независимого ROS-узла. Это позволило разделить функции, такие как получение и предобработка изображений, детекция объектов, выбор цели, восстановление трехмерных координат, трансформация в систему отсчета робота и генерация позы захвата. Обмен данными организован по стандартным топикам с документированными типами сообщений, что обеспечивает совместимость с другими пакетами экосистемы ROS.

Экспериментальная оценка разработанной системы проводилась по трем сценариям, различающимся сложностью фона и условиями освещения. Метрики качества детекции подтвердили устойчивую работу модуля в контролируемых условиях. В базовом сценарии были достигнуты высокие показатели точности и полноты, а также средняя точность детекции 0,81. Однако при усложнении обстановки показатели снижаются, что указывает на чувствительность модели к вариациям освещения и загроможденности кадра.

Точность восстановления трехмерных координат оценивалась на нескольких дистанциях. На расстоянии 0,5 метров средняя ошибка не превысила 1,8 сантиметра, что достаточно для надежного захвата объектов. Однако с увеличением дальности ошибка возрастает, достигая на 1,5 метрах средней величины 7,9 сантиметров. Такой характер погрешности объясняется как физическими ограничениями RGB-D сенсоров, так и неточностью определения центра ограничивающей рамки.

Сравнение моделей семейства YOLOv8 по быстродействию показало, что наиболее легкая версия YOLOv8n обеспечивает обработку 28 кадров в секунду со средней задержкой 35 миллисекунд. Была выбрана модель YOLOv8m, которая обеспечивает приемлемую точность, умеренную вычислительную нагрузку и частоту около 17 кадров в секунду.

Основные ограничения текущей реализации связаны с работой исключительно в симуляции, отсутствием адаптивных стратегий выбора объекта

и фиксированной вертикальной ориентацией схвата. Кроме того, шум карт глубины и ошибки локализации рамки на границах объектов остаются источником погрешностей, которые потребуют дополнительных алгоритмических решений при переходе к реальному оборудованию.

Дальнейшее развитие модуля предполагает перенос на физическую платформу с реальной RGB-D камерой, интеграцию адаптивного планирования ориентации схвата в зависимости от геометрии объекта, расширение набора детектируемых классов и внедрение трекинга для уменьшения вычислительной нагрузки. Предложенная архитектура и полученные количественные оценки создают основу для учебно-исследовательских проектов и построения промышленных прототипов сортировочных комплексов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Macenski S., Foote T., Gerkey B., Lalancette C., Woodall W. ROS 2: Design, architecture, and uses in the wild // Science Robotics. – 2022. – Vol. 7, Issue 66.
- [2] Coleman D., Sucan I., Chitta S., Correll N. Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study // Journal of Software Engineering for Robotics. – 2014. – Vol. 5, No. 1. – P. 3–16.
- [3] Redmon J., Farhadi A. YOLOv3: An Incremental Improvement. – 2018. – URL: <https://arxiv.org/abs/1804.02767>.
- [4] Jocher G. et al. Ultralytics YOLOv8 Documentation. – 2023. – URL: <https://docs.ultralytics.com>.
- [5] Bradski G., Kaehler A. Learning OpenCV: Computer Vision with the OpenCV Library. – Sebastopol: O'Reilly Media, 2008. – 580 p.
- [6] Wong C. C. et al. Implementation of a Real-Time Object Pick-and-Place System Based on ROS // Sensors. – 2023. – Vol. 23, Issue 11. – 5151.
- [7] Koenig N., Howard A. Design and use paradigms for Gazebo, an open-source multi-robot simulator // IEEE/RSJ IROS. – Sendai, 2004. – P. 2149–2154.
- [8] Lin T.-Y. et al. Microsoft COCO: Common Objects in Context // ECCV. – Zurich, 2014. – P. 740–755.
- [9] Huang J., Rathod V., Sun C., et al. Speed/accuracy trade-offs for modern convolutional object detectors // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). – 2017. – P. 7310–7319.
- [10] Reddi V. J., Cheng C., Kanter D., et al. MLPerf Inference Benchmark

// ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA). – 2020. – P. 446–459.

ПРИЛОЖЕНИЕ А

ЛИСТИНГИ ОСНОВНЫХ МОДУЛЕЙ

A.1 Модуль технического зрения (vision_node): основные функции

Листинг 1: Основные функции vision_node

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2D, Detection2DArray,
    ObjectHypothesisWithPose
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2

class VisionNode(Node):
    def __init__(self):
        super().__init__('vision_node')
        self.bridge = CvBridge()
        self.model = YOLO('yolov8n.pt')
        self.conf_threshold = 0.5

        self.subscription = self.create_subscription(
            Image,
            '/rgb_camera/image',
            self.image_callback,
            10)

        self.publisher = self.create_publisher(
            Detection2DArray,
            '/detections',
            10)
```

```

def image_callback(self, msg):
    cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    results = self.model(cv_image, conf=self.conf_threshold)

    detections_msg = Detection2DArray()
    detections_msg.header = msg.header

    for result in results[0].boxes:
        detection = Detection2D()
        hypothesis = ObjectHypothesisWithPose()
        hypothesis.hypothesis.class_id = str(int(result.cls
            [0]))
        hypothesis.hypothesis.score = float(result.conf[0])
        detection.results.append(hypothesis)

        x1, y1, x2, y2 = result.xyxy[0].tolist()
        detection.bbox.center.x = (x1 + x2) / 2.0
        detection.bbox.center.y = (y1 + y2) / 2.0
        detection.bbox.size_x = x2 - x1
        detection.bbox.size_y = y2 - y1

        detections_msg.detections.append(detection)

    self.publisher.publish(detections_msg)

```

A.2 Модуль оценки трехмерных координат (target_3d_estimator): основные функции

Листинг 2: Основные функции target_3d_estimator

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import Point
from cv_bridge import CvBridge

```

```

import cv2
import numpy as np

class Target3DEstimator(Node):
    def __init__(self):
        super().__init__('target_3d_estimator')
        self.bridge = CvBridge()
        self.depth_image = None
        self.camera_info = None
        self.target_center = None

        self.depth_sub = self.create_subscription(
            Image,
            '/rgbd_camera/depth_image',
            self.depth_callback,
            10)

        self.info_sub = self.create_subscription(
            CameraInfo,
            '/rgbd_camera/camera_info',
            self.info_callback,
            10)

        self.target_sub = self.create_subscription(
            Point,
            '/target_center',
            self.target_callback,
            10)

        self.publisher = self.create_publisher(
            Point,
            '/target_3d',
            10)

    def compute_3d_point(self, u, v):

```

```

fx = self.camera_info.k[0]
fy = self.camera_info.k[4]
cx = self.camera_info.k[2]
cy = self.camera_info.k[5]

depth_value = self.get_depth_at_center(u, v, window_size
=5)

if np.isnan(depth_value) or depth_value <= 0:
    return None

X = (u - cx) * depth_value / fx
Y = (v - cy) * depth_value / fy
Z = depth_value

point = Point()
point.x = X
point.y = Y
point.z = Z
return point

```